Free Value Or Else

Steve Downey <sdowney@gmail.com>

Document #: D4270R0
Date: 2026-06-12
Project: Programming Language C++
Audience: LEWG

Abstract

A set of free functions implementing the pattern of optional and expected `value_or` as free functions that produce the correct common reference type that was unfixable for `std::optional::value_or`.

Contents

Changes Since Last Version

— **R0** Initial revision.

1 Comparison table

1.1 Raw pointer with a manual null check

```
Cat* cat = find_cat("Fido");
return cat ? *cat : Cat{"Default"};
```

```
Cat* cat = find_cat("Fido");
return std::value_or(cat, Cat{"Default"});
```

1.2 The value_or truncation pitfall

```
std::optional<std::uint16_t> o = get();
auto v = o.value_or(-1);
// v is uint16_t{65535}
```

```
std::optional<std::uint16_t> o = get();
auto v = std::value_or(o, -1);
// v is int, value preserved
```

1.3 Reference alternative without copies

```
std::optional<Logger&> opt = get_logger();
Logger& l =
    opt ? *opt : default_logger();
```

```
std::optional<Logger&> opt = get_logger();
Logger& l = std::reference_or(
    opt, default_logger());
```

2 Motivation

3 Design

3.1 The `nullable` concept

3.2 Return type computation

3.3 `reference_or`

3.4 `or_invoke` and laziness

3.5 Why free functions

3.6 `expected` interaction

3.7 What about `yield_if`

4 Principles for Reification of Design

5 Implementation Experience

```
template <class T>
concept nullable = requires(const T t) {
    bool(t);
    *(t);
};

template <nullable T, class U,
          class R = std::common_reference_t<std::iter_reference_t<T>, U&&>>
constexpr auto reference_or(T&& m, U&& u) -> R;

template <nullable T, class U,
          class R = std::common_type_t<std::iter_reference_t<T>, U&&>>
constexpr auto value_or(T&& m, U&& u) -> R;

template <nullable T, class I,
          class R = std::common_type_t<std::iter_reference_t<T>,
                                       std::invoke_result_t<I>>>
constexpr auto or_invoke(T&& m, I&& invocable) -> R;
```

6 Proposal

7 Wording

8 Impact on the standard

9 Acknowledgments

10 Document history

— **R0** 2026-06-12 — Initial draft. The free-function follow-on promised in [P2988R12], continuing [P1255R13].

References

[P1255R13] Steve Downey. P1255R13: A view of 0 or 1 elements: views::nullable and a concept to constrain maybes. https://wg21.link/p1255r13, 5 2024.

[P2988R12] Steve Downey and Peter Sommerlad. P2988R12: std::optional<t&>. https://wg21.link/p2988r12, 4 2025.